

ClauseGuard: Agentic Contract Risk Triage

Kartik Pothapragada | Track A - Agentic / Multi-Agent AI | Scenario 2 - Gen AI for Enterprise Documents

1. Problem and domain

Mid-market companies sign hundreds of vendor agreements each year, usually on vendor paper. Legal teams must compare every agreement with an internal playbook: liability caps, breach-notice deadlines, termination rights, data protection, indemnity, and other negotiated positions. Because legal capacity does not scale with volume, the long tail is skimmed and material exposure can remain hidden until an incident.

ClauseGuard triages that queue. It classifies a contract, locates the governing clause text, compares each clause with a versioned playbook, scores exposure, verifies every claim against the source, and either clears the contract or creates an evidence-backed escalation packet for counsel.

Track A is the right fit because this is an accountable decision workflow, not merely question answering. A RAG system (Track D) could retrieve relevant policy and generate a grounded answer, but it would not by itself provide explicit repair routing, a bounded retry budget, an asynchronous human approval gate, or an auditable record of which role made each decision. A single LLM call would also be unable to independently challenge itself or re-extract only one failed clause. The agentic design makes those control points explicit.

2. Approach and algorithm decisions

Orchestration: LangGraph. The workflow is a state machine with conditional edges: intake, extraction, policy assessment, deterministic scoring, verification, escalation, human review, and drafting. LangGraph was chosen over CrewAI and AutoGen because `interrupt()` plus a checkpointer can suspend a run and resume it later or in another process. Its explicit conditional edges also make escalation and repair inspectable business rules. The trade-off is more wiring and less emergent conversation, which is desirable for a safety-sensitive gate.

Six specialized agents instead of one prompt. Intake scopes the work; Extractor quotes source spans; Policy performs the deep language judgement; Risk computes scores; Verifier attacks the result; and Drafter assembles the memo. Merging Policy and Verifier was rejected because a model grading its own output shares the context that produced the error. A separate deterministic Risk node was preferred to an LLM scorer because arithmetic drift or prompt variation could flip a legal-review decision. A per-clause agent was also rejected: fourteen near-identical agents would add cost without adding a meaningful control boundary.

Retrieval: BM25 plus dense search with Reciprocal Rank Fusion. Playbook queries mix exact signals such as “net 15” and “72 hours” with paraphrases. BM25 handles literals but misses semantic matches; dense retrieval handles paraphrase but can blur numbers. RRF was chosen over score interpolation because BM25 and cosine scores are not naturally comparable and interpolation would add a corpus-specific tuning parameter. Retrieval remains a tool used by the agents, not the primary deliverable.

Grounding and verification. The Extractor returns quoted text, while Python computes offsets by locating that quote in the original document. The Verifier checks exact or near-exact span grounding, numeric consistency, and then LLM entailment only for grounded non-compliant findings. A missing required clause is assessed as an absence finding rather than skipped. Ungrounded quotes are repaired once or escalated after a maximum of two repair passes. These decisions were preferred to trusting model-reported offsets, fuzzy matching alone, or an unbounded retry loop.

3. Results and error analysis

The following results use the deterministic offline rule-engine backend. They measure pipeline and orchestration behavior, not LLM quality; the backend exists for reproducibility without credentials. Severity labels are synthetic because severity is defined by this company's playbook.

Metric	Synthetic: 24 docs, 233 judgements	Adversarial: 8 docs, 78 judgements
Severity accuracy	0.880	0.872

Metric	Synthetic: 24 docs, 233 judgements	Adversarial: 8 docs, 78 judgements
Dangerous-miss rate	**0.000 (0/120)**	**0.140 (6/43)**
False-alarm rate	0.093	0.000
Absence detection recall	1.000 (7)	1.000 (2)
Groundedness rate	1.000 (233)	1.000 (78)
Escalation recall / precision	1.000 / 0.826	0.875 / 1.000

Dangerous misses matter more than aggregate accuracy: a false alarm costs review time, while a real risk graded safe can create major liability. The strongest error signal is the adversarial liability result: the rule engine missed all 6 of 6 deliberately awkward liability clauses, grading each MINOR when the gold label was UNACCEPTABLE. The failure used a double negative and displaced the cap into a defined term, for example language saying that nothing imposes a ceiling on the customer's obligations. Regex-style rules cannot reliably interpret that construction. This measured weakness is the reason to put an LLM in the Policy agent, while retaining deterministic controls around it.

The escalation gate still caught 7 of 8 adversarial contracts, but often because other clauses triggered it. That is useful defence in depth, but not a substitute for correctly identifying the liability problem. Retrieval ablation on 32 labelled queries showed hybrid recall@1 of 0.750 and MRR@3 of 0.839, ahead of BM25 (0.719, 0.760) and dense-only (0.688, 0.812) in the top-1 regime. Hybrid lost both adversarial retrieval queries because both legs ranked the same distractor first; fusion compounded correlated error. Dense-only had better recall@3 (0.969 versus 0.938), so top-k policy retrieval is a clear next experiment.

4. Production and limitations

Production consideration: playbook versioning. The playbook is versioned data, and each assessment records its rule ID and playbook version. A legal update can therefore change future behavior without retraining. In production, a playbook change should emit an event that re-scores affected cached extractions and marks older memos for review. This avoids re-running expensive extraction when only the policy threshold changed. The human gate should use a durable database checkpoint rather than the in-memory demo saver, with access control, audit retention, and monitoring for escalation volume and verifier failures.

Deployment-blocking limitation: synthetic severity labels. Extraction can be compared with public clause annotations, but no public dataset labels severity against this company's private playbook. Before deployment, several hundred clauses should be independently labeled by at least two counsel, with inter-annotator agreement reported. This would establish the real ceiling for MATERIAL versus MINOR judgements and support calibrated evaluation. Other known gaps are scanned/PDF ingestion, conditional playbook rules, and cross-document review of amendments and order forms.